

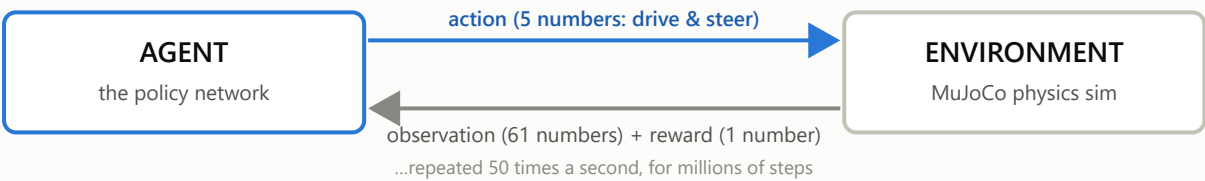
Reinforcement Learning, taught by a robot carrying two cylinders

Everything below happened for real, in simulation, on your laptop: a wheeled robot named MIRTE learned — from nothing but trial, error, and a score — to carry a wobbly stack of two cylinders across a 6.5-meter obstacle course. This document teaches you the ideas, then shows you the campaign.

1 · What reinforcement learning is

Most machine learning you've heard of is **supervised learning**: you show the model a million labeled examples ("this photo is a cat") and it learns to imitate the labels. But nobody can label "the correct motor command for every possible situation of a robot balancing a stack." There is no answer key.

Reinforcement learning (RL) replaces the answer key with a **reward** — a running score. The learner (called the **agent**) tries actions, the world (the **environment**) responds, and a number tells it how well things are going. Behaviors that lead to more total reward get reinforced; ones that lead to disaster fade. Nobody ever tells the agent *how* to drive — it discovers that itself, across millions of attempts.



2 · The cast, in this project's terms

RL CONCEPT	IN GENERAL	IN THE MIRTE EXPERIMENT
Agent / policy	The decision-maker: a function from what-I-see to what-I-do	A small neural network (two hidden layers) driving the robot
Environment	The world that reacts and scores	A MuJoCo physics simulation: robot, payload, pillars, doorway, stairs
Observation	What the agent senses each step	61 numbers: 32 lidar rays, 3 ground rangefinders, stack-lean direction & rate, velocities
Action	What the agent controls	5 numbers: forward/sideways/turn commands and wrist control
Reward	The score after every step	Progress toward the goal, small time penalty, -10 for dropping/stalling, +80 for delivery
Episode	One attempt, start to end	One course run: max 2,500 steps; ends on delivery, drop, crash, or stall

THE ONE-SENTENCE VERSION

RL = **learning a behavior by maximizing a score through trial and error** — the same way you learned to ride a bicycle: nobody handed you the equations, you just fell until you didn't.

How the learning actually happens

3 · The policy is a probability, not a reflex

During training the network doesn't output one action — it outputs a *distribution* (a mean and a spread σ for each of the 5 controls) and **samples** from it. That randomness is **exploration**: the agent deliberately jitters its behavior to stumble onto better moves. As training matures, σ shrinks — it commits to what works. At evaluation time we take the mean (**deterministic** mode): pure skill, no dice. A hard-won lesson: when one fine-tune's σ grew (0.37→0.48, run v18), performance collapsed — noise dropped the stack faster than learning improved it.

4 · Rewards become returns; a critic estimates them

The agent doesn't chase instant reward — reaching the goal pays +80 *fourteen hundred steps from now*. It maximizes the **return**: the sum of all future rewards, each discounted by γ per step (here $\gamma = 0.995$, meaning "care about rewards ~200 steps ahead"). A second network, the **value function (critic)**, learns to predict the return from any state. The difference between what actually happened and what the critic predicted is the **advantage** — "was that action better or worse than expected?" — and that is the actual learning signal.

5 · PPO, in plain words

PPO (Proximal Policy Optimization) is the workhorse algorithm used here (via the Stable-Baselines3 library). Its loop:

- **Collect**: run the current policy in **12 parallel simulations**, gathering ~25k steps of (observation, action, reward) experience per batch.
- **Judge**: compute each action's advantage using the critic.
- **Nudge**: update the network to make above-average actions more likely — but **clip** the update so the new policy can't jump far from the old one in a single step.

That clip is the "proximal" part — why PPO is stable enough to run unattended overnight: a bad batch can nudge the policy off course, never launch it off a cliff.

LEARNING RATE — THE LADDER'S MASTER KNOB

The learning rate (lr) scales every update. This project fine-tuned **generation after generation** (v13→v17→v19→...), and the single most valuable schedule discovered was: **halve the lr each generation as the policy matures**. lr 1e-4 was right when the policy succeeded 0–30% of the time; at 55%+ that same lr destroyed precision (the v18 collapse). By v31 the lr was 5e-6 — twenty times gentler.

6 · Supporting machinery you'd meet in any real RL project

PIECE	WHAT IT DOES	WHY IT MATTERS HERE
VecNormalize	Rescales every observation to mean 0, spread 1, using running statistics	Lidar meters and lean-rates live on different scales; networks learn poorly from raw mixed units
Checkpoints	Snapshot of the policy saved every ~500k steps	Best policy is often <i>mid-run</i> , not the final one — v30's best was at 3M of 8M
Curriculum	Start some episodes mid-course instead of at the start line	Lets the "reaching the goal is good" signal propagate backward through a long course
Eval protocol	Scripted scoring, separate from training	60 deterministic episodes, judged by median (page 4)

One robot, one course, one stubborn payload

7 • The task

MIRTE starts at $x = -3.3$ m and must deliver at $x = +3.2$ m. In between, in order: a **field of pillars** to weave through, a **doorway** 1.0–1.5 m wide in a dividing wall, and a **low staircase** (up, plateau, down). The floor is scattered with bumps. Every episode randomizes the layout — the policy must learn the *skill*, not memorize one map.

The payload is the villain: **two cylinders stacked on top of each other**, standing loose in a 10 cm-walled tray that hangs from a 2-axis powered gimbal on the robot's gripper. Physically, that's an inverted pendulum riding on a pendulum. Any jolt — a bump, hard braking, a clipped pillar — and the top cylinder topples: **episode failed**. A hard project rule kept the science honest: **no caging the stack**. Tall walls or clamps that make dropping impossible would trivially "solve" the task by deleting it.

8 • Reward design is where RL projects are won and lost

The reward function went through wars, and each war taught a classic RL lesson:

WHAT WAS TRIED	WHAT THE AGENT ACTUALLY LEARNED	THE FIX
Time penalty (hurry up)	Rushed the stairs at 1+ m/s, dropping constantly	Speed-shaping only in the stair zone
Progress rewards stacking up	Froze at the doorway, farming safe reward forever	Stall rule: 400 steps without progress ends the episode at -10
Higher penalties for drops	Timid crawling everywhere, still dropped	Physical fixes: speed cap, gentler slew limits, better tray

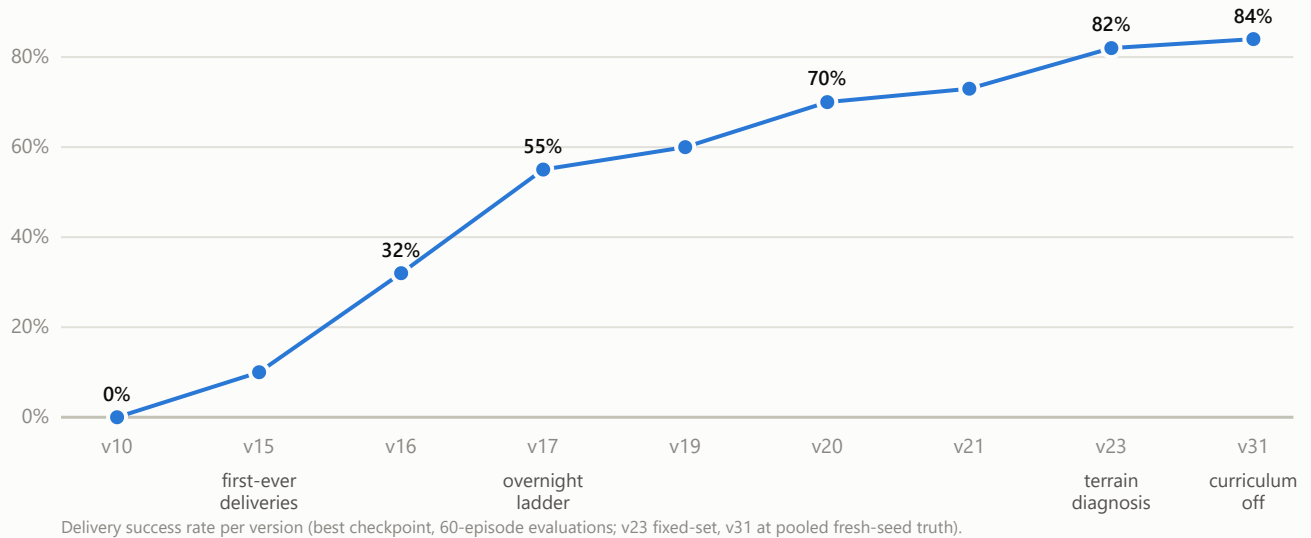
THE PROJECT'S MOST-REPEATED LESSON

Hard physical constraints always worked; soft reward nudges always backfired. The agent optimizes what you *score*, not what you *mean* — if a loophole exists, RL will find it. Making bad behavior *impossible* beats making it *expensive*.

9 • How you know it's actually working: evaluation discipline

- **Never trust the training curve.** One version (v2) had beautiful loss curves and scored 0/20 on the real task. Always run scripted evaluations.
- **60 episodes, report the median.** Single episodes swing wildly; means get dragged by outliers.
- **Fixed seeds vs fresh seeds.** Evaluating on the *same* 60 randomized layouts every time makes versions comparable — but a policy can get "lucky" on that particular set. The truth check is a *fresh* set it has never been measured on. This distinction produced the project's biggest plot twist (page 4).
- **Diagnose before changing.** A *failure map* — logging where and how every failure happened — preceded every real breakthrough. Aggregate scores tell you *that* it fails; maps tell you *why*.

From 0% to ~85%: every gain had a cause



10 · The levers that moved it — and the ones that didn't

LEVER (RL CONCEPT)	RESULT
Curriculum learning — spawn mid-course so goal-value flows backward	Enabled the first completions (v15)
Generation ladder — fine-tune from best checkpoint, halve lr each time	10% → 55% → 70% → 73% across four nights
Diagnosis-driven environment fix — failure stats showed floor bumps caused most drops; reduced 12→6	73% → 80% overnight, zero retraining
Train under a speed cap (after zero-shot capping failed)	Adapted fine; drops at the cap stopped
Bigger action space — give the policy the robot's arm (5→7 actions) to catch drops	Failed 3 separate times — arm motion shook the pendulum worse
Train on the graded task — v31 turned curriculum off; failure map showed 10/12 failures in the course's front half, which scattered spawns under-trained	82% → 84% pooled — the first true gain in three generations

THE 90% PLOT TWIST — A STATISTICS LESSON

One checkpoint scored **54/60 = 90%** on the standard evaluation set — goal reached! Then the honesty check: on a **fresh** 60 layouts it had never been measured on, the same checkpoint scored 44/60. Neither number is fake — the first was a favorable draw. Pooling many independent sets put every top checkpoint at **~82%**. Lesson: **a single evaluation is a sample, not a truth**. Confirm records on data that had no vote in selecting the record-holder.

What this experiment actually teaches

11 · Learning ceilings vs physics ceilings

By the end, every honest lever was exhausted and the score plateaued at ~82–85%. The failure map showed the residual: drops while weaving pillars at moderate speed — the tall pendulum-stack physics, not a lack of learning. That's a **physics ceiling**: past it, only changing the *robot* (a real clamp, a different payload spec) helps — and shake-tests with the policy switched off proved the grip geometry didn't matter either. Knowing *which* ceiling you've hit is the difference between a good next experiment and a wasted week of GPU time.

12 · The honest-metrics toolkit (transferable to any ML work)

- **Fixed benchmark for comparisons, fresh data for claims.**
- **Median over mean** when single runs can swing the number.
- **Diagnose, then intervene** — one targeted change at a time, so causes stay attributable.
- **Stop rules decided in advance** — "no judging before 50% through; stop after 2 consecutive declines" — so impatience doesn't drive decisions.
- **Never let metric easing inflate results** — when the course was eased (wider door), history was re-read under the new protocol, footnoted.

13 · Caveat: this is simulation

Everything here ran in MuJoCo, a physics simulator. Real-robot transfer (**sim2real**) is its own discipline: simulators are optimistically clean, so policies typically need domain randomization (deliberately varied physics) to survive reality. For a LinkedIn-honest summary: "*trained in simulation*" is the accurate phrase.

14 · Glossary — the terms you now own

Agent	The learner/decision-maker	PPO	Stable policy-gradient algorithm; clipped updates
Policy	Its behavior: observations → actions (a neural net)	Critic / value fn	Predicts return; its error is the learning signal
Environment	The world that reacts and scores	Advantage	"Better than expected?" per action
Reward	Per-step score; the only supervision RL gets	Learning rate	Update step size; halve it as skill matures
Return	Total (discounted) future reward	Curriculum	Easier/rearranged starts that scaffold learning
γ (gamma)	Discount: how far ahead the agent cares	Checkpoint	Saved policy snapshot; best is often mid-run
Episode	One attempt, reset to done	Fine-tune	Continue training from a checkpoint, gently
Exploration	Trying non-best actions to find better ones	Sim2real	The gap between simulator success and reality

Go deeper: Sutton & Barto, *Reinforcement Learning: An Introduction* (the standard text, free online) · OpenAI Spinning Up (spinningup.openai.com — PPO explained properly) · Stable-Baselines3 docs (the library used here) · and this repo itself: `mirte_gimbal_env.py` is a complete, battle-tested custom environment.